

Clinic Management System

Server & Infrastructure Report

What runs the system, where it lives, and how it is built —
explained for everyone, detailed for engineers.

Item	Details
Report date	16 June 2026
Live website	https://clinic.nalexustechnologies.com
Server	Cloud VPS · Ubuntu 24.04 LTS · IP 31.97.190.216
Prepared from	A live, read-only inspection of the production server and database.
Confidentiality	Passwords and secret keys are intentionally excluded from this document.

This report is written in two voices on every topic: a plain-language explanation (green) anyone can follow, and a technical specification (blue / tables) for engineers.

1. The System at a Glance

In plain words: this is a complete software system that helps a clinic run its daily work — registering patients, booking appointments, doctor consultations, prescriptions, pharmacy, billing, reports, and AI assistance for doctors. It is a live website that staff open in a browser. Behind that website sits a powerful rented computer (a server) in a data centre that is online 24/7, keeps everyone's data safe, and does all the heavy thinking.

Think of it like a restaurant. The **website you see** is the dining room. **Nginx** is the front-door host who greets every guest and checks they came through the secure (HTTPS) entrance. **Gunicorn + Flask** are the kitchen where every order is actually cooked. The **database** is the store-room where all ingredients and records are kept. Each part has one job, and together they serve every request quickly and safely.

Question	Short answer
Where is it deployed?	Cloud VPS server, IP 31.97.190.216, domain clinic.nalexustechologies.com
What is the backend?	Python (Flask) REST API run by Gunicorn, kept alive by the operating system
What database is used?	Percona Server for MySQL 8.4 (a hardened, enterprise build of MySQL)
What is the frontend?	A Flutter web app (compiled to a fast static website) served by Nginx
How is it secured?	HTTPS/TLS certificate, firewall via localhost-only services, fail2ban, isolated app user
Is it live now?	Yes — running continuously; the server has been up 108 days

2. The Server (the physical home of the system)

In plain words: instead of buying a computer and keeping it in the office, the clinic rents a professional-grade computer in a secure data centre. It is always on, always connected to the internet, has battery/Generator backup, and is far more reliable than an office PC. This rented computer is called a **VPS** (Virtual Private Server). Ours is a fast, modern machine with 4 processor cores and 15 GB of memory — comfortably more than this clinic system needs, so it stays quick even when busy.

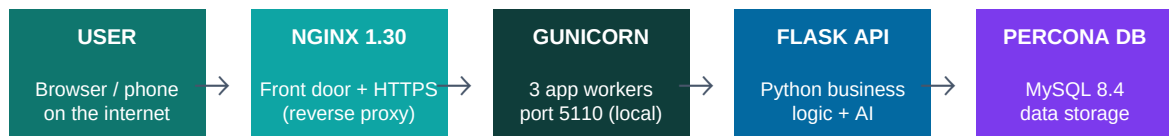
Technical specification (read live from the server):

Component	Specification
Server type	Virtual Private Server (VPS), hostname srv951538
Public IP address	31.97.190.216
Operating system	Ubuntu 24.04.3 LTS (Noble Numbat) — long-term-support Linux
Kernel	Linux 6.8.0-90-generic (64-bit)
Processor	AMD EPYC 9354P (4 vCPU cores allocated)
Memory (RAM)	15 GiB total + 2 GiB swap
Storage	193 GB SSD (about 20% used)
Availability	Up 108 days continuously at time of report; load average < 0.2 (very light)

Why this matters: a long uptime and a very low load average mean the system is stable and nowhere near its limits — there is large headroom for more clinics, users, and data.

3. How a Request Travels (architecture)

In plain words: every time a staff member clicks something, their request makes a short, well-guarded journey and comes back with an answer in a fraction of a second. Here is that journey, left to right:



Step	Component	What it does
1	User device	A browser or phone opens clinic.nalexustechnologies.com over the secure internet.
2	Nginx (reverse proxy)	The 'front door'. Forces HTTPS, serves the website files, and forwards data requests (/api/...) inward. Shields the application from the open internet.
3	Gunicorn	The application server. Runs 3 parallel worker processes so several users are served at once. Listens only on the server's own internal address (127.0.0.1:5110).
4	Flask API	The 'brain'. Python code that checks permissions (JWT login), applies clinic rules, runs AI features, and reads/writes data.
5	Percona / MySQL	The 'memory'. Safely stores every patient, appointment, payment, prescription, and record, and returns exactly what was asked for.

A key security point in plain words: only Nginx is exposed to the public internet. The application (Gunicorn/Flask) and the database both listen on the server's private internal address only, so the outside world can never talk to them directly — every request must pass through the guarded front door first.

4. The Backend (the engine room)

In plain words: the backend is the part nobody sees but everybody depends on. It enforces the rules ('a receptionist can book appointments but not prescribe medicine'), keeps logins secure, calculates revenue, and talks to the database. It is written in Python using a well-established framework called Flask, and it is kept permanently running by the operating system — if it ever crashed, the server restarts it automatically within seconds.

Item	Detail
Language	Python 3.12.3
Web framework	Flask 3 (REST API)
Application server	Gunicorn — 3 worker processes, bound to 127.0.0.1:5110, 120s timeout
Process manager	systemd service 'clinic-backend.service' (auto-restart on failure, starts on boot)
Runs as	Non-root user 'nalexus', group 'www-data' (limited privileges = safer)
Environment	FLASK_ENV=production; isolated Python virtual-env at /home/nalexus/clinic/backend/venv
API design	18 route groups under /api/* (auth, doctors, pharmacy, payments, reports, clinical-ai, ...)
Security model	JWT login tokens (24h), 7 user roles, every change written to an audit log

For engineers: the app uses the Flask application-factory pattern with 18 blueprints, a layered routes → services → models design (SQLAlchemy ORM + Flask-Migrate), Flask-JWT-Extended for stateless auth, and an after-request hook that records every successful mutating call to an audit_logs table. Run target is `run:app` behind Gunicorn, supervised by systemd with `Restart=always`.

5. The Database (where everything is remembered)

In plain words: the database is the system's permanent memory. Every patient, doctor, appointment, prescription, payment and report is stored here in neatly organised tables, like a set of perfectly maintained ledgers. We use Percona Server — a professional, extra-reliable version of the world's most popular open-source database, MySQL. It lives on the same server but is locked to internal access only, so no outsider can reach it.

Item	Detail
Engine	Percona Server for MySQL 8.4.7 (drop-in MySQL-compatible, enterprise-grade)
Database name	clinic
Access	Local only (127.0.0.1:3306) — not reachable from the internet
Character set	utf8mb4 (full Unicode — supports any language and emoji)
Login user	dedicated 'clinic' account with password (excluded from this report)
Total size	≈ 1.44 MB across 20 tables (live operational + sample data)
Schema management	Flask-Migrate (versioned migrations) — schema changes are tracked and repeatable

Live table inventory (the 20 tables that make up the system, with approximate row counts):

Table	Rows	Table	Rows
users	24	patient_vitals	15
clinics	1	patient_reports	15
departments	7	consultation_drafts	15
doctors	2	prescriptions	15
assistants	2	prescription_medicines	30
patients	15	prescription_lab_tests	15
appointments	15	pharmacy_items	15
payments	15	pharmacy_sales	15
subscription_plans	4	pharmacy_sale_items	15
clinic_subscriptions	1	audit_logs	15

The 24 user accounts already span all seven roles (super_admin, clinic_admin, doctor, assistant, receptionist, pharmacy, and patient), confirming every portal has at least one working login on the live system.

6. The Frontend (what people actually see)

In plain words: the frontend is the screen the staff use — the buttons, menus, tables and forms. It is built with Flutter, Google's modern app technology, and compiled into a fast website. Because each role sees its own tailored portal, a receptionist, a doctor and a pharmacist all open the same address but get completely different, relevant screens.

Item	Detail
Technology	Flutter (Dart) — compiled to an optimised static web app
Served by	Nginx directly from /home/nalexus/clinic/frontend (very fast, cached)
Talks to backend	Via HTTPS calls to /api/* on the same domain

Item	Detail
State / login	Provider state management; JWT token kept in secure storage; auto-logout on expiry
Portals	Super Admin, Clinic Admin, Doctor, Assistant, Receptionist, Pharmacy, Patient
Exports	Built-in PDF, CSV and JSON export for reports

For engineers: single-page Flutter web build with client-side role guards (`route_guard`), a central `ApiService` injecting the bearer token and handling 401 → forced re-login, and conditional web/io implementations for storage and file export. Nginx uses `try_files $uri /index.html` so client-side routing works on refresh.

7. Security & Safety

In plain words: the system is built so that data stays private and the service stays available. Connections are encrypted, the sensitive parts are hidden from the internet, and the server defends itself against intruders automatically.

- **Encrypted traffic (HTTPS/TLS):** a valid Let's Encrypt certificate secures every connection; plain HTTP is automatically redirected to HTTPS.
- **Hidden internals:** the application (port 5110) and database (port 3306) listen on the internal address only — the public can reach Nginx and nothing else.
- **Strong logins:** passwords are stored only as secure hashes (never as plain text) and access is controlled by JWT tokens with a 24-hour expiry.
- **Role-based access:** seven roles ensure each user can only do what their job allows.
- **Full audit trail:** every create/update/delete is logged with who, what, and when.
- **Intrusion defence:** fail2ban is active, automatically blocking repeated malicious login attempts (e.g. against SSH).
- **Least privilege:** the app runs as a restricted user, not as the all-powerful root account.
- **Automatic recovery:** systemd restarts the backend within seconds if it ever fails

For engineers: TLS 1.2/1.3 via certbot with HSTS-ready options-ssl-nginx.conf and a custom dhparam; reverse-proxy isolation; non-root systemd unit with Restart=always; secrets injected through an .env file readable only by the service user (mode 0640); CORS locked to the production origin only.

8. The AI Features (briefly)

In plain words: the system includes AI help for doctors — it can summarise a patient's history, draft consultation notes, and flag things worth checking. There are also educational 'risk score' tools (for diabetes, heart, stroke, breast-cancer screening) that turn simple measurements into a risk percentage. These are assistants, not decision-makers: a doctor always reviews and confirms.

AI capability	How it works
Clinical documentation AI	Sends the patient's clinic data to an OpenAI model (configured: gpt-5.2) and returns structured drafts; if the AI is unreachable it falls back to a built-in offline engine.
Disease risk tools	Four scikit-learn machine-learning models run directly on the server, turning numeric inputs (age, BMI, glucose, etc.) into a risk probability and a low/moderate/high level.
Safety	Every AI output carries a notice that a licensed clinician must verify it before use.

Note on AI provider: the clinical assistant is configured to call OpenAI's API. The provider is a single configuration value (CLINICAL_AI_PROVIDER / CLINICAL_AI_MODEL), so it can be switched — for example to Anthropic's Claude models — with a small, isolated change, since all provider calls are contained in one service module.

9. Summary for Evaluators

This is a professionally deployed, production-grade clinic platform. It runs on a modern Ubuntu cloud server with a clean, secure architecture: Nginx as the public gateway, Gunicorn + Flask as a resilient Python backend supervised by the operating system, and a hardened Percona/MySQL database — all reachable only through one encrypted front door. The frontend is a fast Flutter web app with role-specific portals, and the system layers in both classical machine-learning and large-language-model AI for doctors, with explicit human-in-the-loop safety. The technology choices are mainstream, well-supported, and scalable, and the live server shows ample spare capacity for growth.

Layer	Technology (verified live)
Domain / TLS	clinic.nalexustechnologies.com · Let's Encrypt HTTPS
Server	Ubuntu 24.04 LTS VPS · AMD EPYC · 4 vCPU · 15 GB RAM · IP 31.97.190.216
Web / proxy	Nginx 1.30.1 (HTTP→HTTPS, static frontend + /api proxy)
App server	Gunicorn (3 workers) via systemd · Python 3.12
Backend	Flask 3 REST API · JWT · 7 roles · audit logging
Database	Percona Server for MySQL 8.4 · 'clinic' · 20 tables · utf8mb4
Frontend	Flutter web (static) · per-role portals · PDF/CSV/JSON export
AI	OpenAI gpt-5.2 (configurable) + 4 scikit-learn risk models

Prepared on 16 June 2026 from a direct, read-only inspection of the production server and database. Credentials and secret keys are deliberately omitted.